

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Requirements Specification

Event Management System(EMS)

Prepared by Prajin Ravi Shankar, Rakshan R, Palash Agarwal, Veladi Yathindra Deveshwar

PES University

06-10-2026



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions, acronyms, and abbreviations

1.4 References

1.5 Overview

2. Overall description

2.1 Product perspective

2.2 Product functions

2.3 User characteristics

2.4 Constraints

2.5 Assumptions and dependencies

3. Specific requirements

3.1 External Interface

3.2 Functional Requirements

3.3 Non-Functional Requirements

3.4 Design Constraints

Appendixes

Index

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Revision History

Name	Date	Reason For Changes	Version
Prajin Ravi Shankar	06-10-2026	Prepared the Introduction, Overall Description and Project Description.	1.0
Rakshan R	06-10-2026	Prepared the External Interface Requirements and Analysis Models (use case diagram, use case summary and ER diagram).	1.1
Palash Agarwal	06-10-2026	Added the Functional Requirements (System Features F1 to F8) and Appendix B: Field Layouts.	1.2
Veladi Yathindra Deveshwar	06-10-2026	Added the Non-Functional, Security and Business Rules sections, Other Requirements, Appendix A: Glossary and Appendix C: Traceability Matrix.	1.3

1. Introduction

1.1 Purpose

This document describes the software requirements for version 1.0 of the Event Management System (EMS). EMS is a web application where organizers publish events and attendees find and register for them. The SRS covers the whole product: the web client, the server, the database and the email integration. It will be the starting point for our design document (SAD) and our test plan (STP).

1.2 Intended Audience

This SRS is meant for four groups of readers:

- ***Developers (our team):*** *use Sections 2, 5 and 6 to know what has to be built.*
- ***Testers:*** *write test cases from the numbered requirements and the traceability matrix in Appendix C.*
- ***Course instructors and reviewers:*** *check that the requirements are clear, measurable and traceable.*
- ***Future maintainers:*** *understand the intended behaviour and business rules.*

Sections 1 and 2 give the background, Section 3 lists the interfaces, Section 4 has the use case and data models, Section 5 lists the functional requirements feature by feature, Section 6 covers the non-functional and security requirements, and the appendices hold the glossary, field layouts and traceability matrix.

1.3 Product Scope

Colleges usually run events through posters, Google Forms, spreadsheets and WhatsApp groups. Seat counts get lost, people register twice, and organizers have no clear attendance numbers afterwards. EMS puts all of this in one place: creating events, setting capacity, taking registrations, informing attendees, marking attendance and producing reports. Typical events are fests, workshops, seminars, club activities and guest lectures.

The goals for version 1.0 are:

1. *An organizer should be able to publish an event and open registration in under 5 minutes.*
2. *An event must never be booked beyond its capacity.*
3. *Attendees should get confirmations, reminders and updates by email without anyone sending them manually.*
4. *Organizers should see attendance figures right after the event.*

Out of scope for version 1.0: online payments and ticket pricing, native mobile apps, live streaming, seating charts, integration with the university ERP, and multi-language support.

1.4 References

- *IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.*
- *ISO/IEC/IEEE 29148:2018, Requirements engineering.*
- *PES University, Software Engineering (UE24CS341A): Mini-Project Guidelines and Problem Statements (Section F).*
- *OWASP Application Security Verification Standard 4.0 and OWASP Top 10 (2021).*
- *Software Architecture & Design Specification (SAD) v1.0 and Software Test Plan (STP) v1.0 of this project.*

Project Description

Problem statement 7 asks for a system to manage and organize events with registrations. We are building it as a new project (not extending a sample project) using the MERN stack: MongoDB, Express, React and Node.js, which is one of the two stacks the problem statement allows.
A visitor can browse the published events without logging in. To register, a person creates an account and becomes an Attendee. An Organizer (a club or faculty member) creates an event with date, venue, capacity and registration deadline. The event stays hidden until an Admin approves it. After approval it appears in the public listing, attendees register, and when the event is full new attendees go on a waitlist. If someone cancels, the first person on the waitlist gets the seat automatically. On the day of the event the organizer checks people in using the registration ID or the QR code, and afterwards looks at the attendance report or downloads the attendee list as CSV.
Customer. *We do not have an external customer yet. The people who would use a system like this are college clubs, department event committees and faculty who organize talks and workshops. [If you have spoken to a club, a professor or an event cell that wants this, name them here in one line.]*
Typical users. *Students and staff (mostly on phones, basic computer skills), club and faculty organizers (moderate use, need simple forms), and one to three administrators who approve events and manage users.*
What the user will be able to do (functional features):

- *Create an account, log in, and reset a forgotten password.*
- *Browse, search and filter upcoming events and see how many seats are left.*
- *Register for an event and receive a unique registration ID with a QR code.*
- *Join a waitlist when an event is full and get promoted automatically when a seat frees up.*
- *Cancel a registration and see the status of all own registrations.*
- *Create, edit and cancel events (Organizer), with every new event approved or rejected by an Admin.*
- *Receive email confirmations, 24-hour reminders, and notices about changes or cancellations.*
- *Check attendees in by registration ID or QR code.*
- *View registration and attendance statistics and export the attendee list as CSV.*
- *Manage users, roles and event categories (Admin).*

2. Overall Description

2.1 Product Perspective

EMS is a new, self-contained web application. It does not replace or connect to any existing campus system. It has a React client in the browser, a REST API server built with Node.js and Express, a MongoDB database, and an outside SMTP service for sending email.

External entity	How it interacts with EMS
Web browser (Visitor, Attendee, Organizer, Admin)	Uses the web client over HTTPS
SMTP email service	Receives emails from EMS: confirmations, reminders, notices, password reset links

2.2 Product Functions

- **F1 User accounts and authentication:** sign up, log in, password recovery, role-based access for Attendee, Organizer and Admin.
- **F2 Event creation and approval:** organizers create and manage events; each new event waits for Admin approval.

- **F3 Event discovery:** anyone can browse, search and filter published events and see remaining seats.
- **F4 Event registration and waitlist:** register, join the waitlist when full, cancel, and track status, without ever exceeding capacity.
- **F5 Notifications:** email confirmations, reminders, waitlist promotions, event changes and cancellations, with delivery status recorded.
- **F6 Attendance check-in:** organizers mark attendance by registration ID or QR code.
- **F7 Reports and dashboards:** per-event statistics and CSV export for organizers, platform summary for Admins.
- **F8 Administration:** manage users, roles and event categories.

2.3 User Classes and Characteristics

User class	Characteristics	Importance
Visitor	Not logged in. Can browse and search events. Any technical level, occasional use.	Medium
Attendee	Students and staff. Registers and manages own registrations. Heavy use around events, basic skills, mostly on phones.	High
Organizer	Club or faculty members. Creates events, checks people in, views reports. Moderate use, needs simple forms.	High
Admin	One to three people. Approves events, manages users and categories. Highest privilege, busy during event season.	High

2.4 Operating Environment

- Client: the latest two versions of Chrome, Firefox and Edge on Windows, macOS, Android and iOS.
- Server: Node.js 20 LTS on Linux (Ubuntu 22.04 or later), or Windows during development. Database: MongoDB 7.x.
- Network: the server needs internet access to reach the SMTP service.

2.5 Design and Implementation Constraints

- The technology stack is MERN (MongoDB, Express, React, Node.js).
- Code is kept in the team GitHub repository and every member has to commit.
- Only free tiers or open-source tools can be used (for example Mailtrap or Gmail SMTP for email).
- The security requirements in Section 6.3 must be met, including HTTPS and authorization checks on the server.

- The work follows the course schedule: Part-1 documents first, then sprint-based implementation.

2.6 Assumptions and Dependencies

- Users have a valid email address and a modern browser with JavaScript turned on.
- An SMTP service is available. If it is down, emails are retried (EMS-F-025) and the rest of the system keeps working.
- The first Admin account is created by seeding the database. New Organizers are made by an Admin promoting an Attendee (EMS-F-031).
- All events are free, so no payment handling is needed.
- Expected load is a campus-sized one: up to 1,000 users and 200 events.

3. External Interface Requirements

3.1 User Interfaces

The web client provides these main screens:

Screen	Used by	Key elements
Home / Event list	Everyone	Search box, category/date/availability filters, event cards with remaining seats, pagination
Event detail	Everyone	Event details, remaining seats, Register / Join waitlist button for Attendees
Login / Register / Reset password	Visitor	Forms with inline validation messages
My Registrations	Attendee	Status list, cancel button, QR code
Create / Edit Event	Organizer	Event form with field-level errors
Event Dashboard and Check-in	Organizer	Counts, attendance %, registration ID entry or QR scan, CSV export
Admin Console	Admin	Pending approvals, users and roles, categories, platform summary

Common rules for every screen: a navigation bar shows the logged-in user and role; error messages appear next to the field they belong to, in plain language; actions that cannot be undone (cancel event, cancel registration, deactivate user) ask for confirmation; the layout works from 360 px to 1920 px wide.

3.2 Software Interfaces

Interface	Version	Purpose and data exchanged
MongoDB	7.x	Stores users, events, categories, registrations, notification logs and audit logs
SMTP email service	SMTP over TLS	Outgoing mail (recipient, subject, body); returns accepted or failed
Server REST API	HTTP/1.1, JSON	Communication between the web client and the server

3.3 Communications Interfaces

- Browser to server: HTTPS (TLS 1.2 or higher), JSON bodies, UTF-8.
- Server to SMTP: SMTP with STARTTLS on port 587. Credentials come from environment variables.
- Server to MongoDB: MongoDB wire protocol. The database port must not be open to the internet.

3.4 Hardware Interfaces

The organizer can scan a registration QR code using a camera on localhost or HTTPS, decode an uploaded QR image, or enter the registration code manually.

The final demo uses localhost HTTP. Production HTTPS and TLS requirements remain in section 6.3. MongoDB must support transactions through a replica set or Atlas. The SMTP transport is configurable.

See the recorded implementation deviations.

Analysis Models

4.1 Use Case Diagram

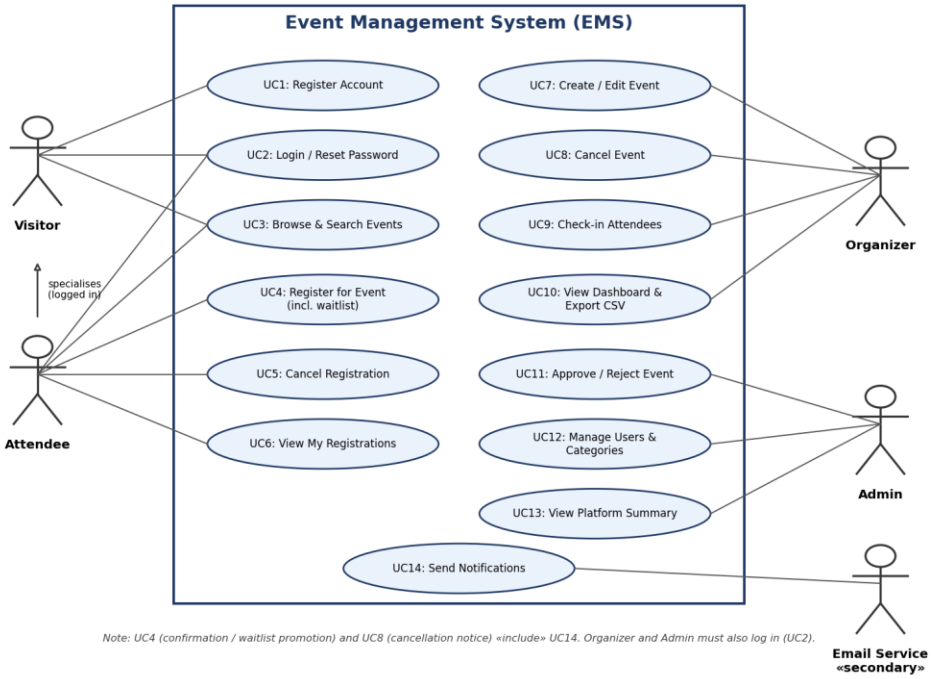


Figure 1: UML use case diagram of EMS

4.2 Use Case Summary

Use case	Actor(s)	Brief description	Related requirements
UC1 Register Account	Visitor	Create an Attendee account	EMS-F-001
UC2 Login / Reset Password	All users	Authenticate; recover forgotten password; lockout rules	EMS-F-002, 003, 005
UC3 Browse & Search Events	Visitor, Attendee	List, search, filter, open event details	EMS-F-013 to 015
UC4 Register for Event	Attendee, Email Service	Register, or join waitlist if full; confirmation email	EMS-F-016 to 018, 021, 022
UC5 Cancel Registration	Attendee	Cancel; first waitlisted attendee is promoted	EMS-F-018, 019
UC6 View My Registrations	Attendee	See all registrations and statuses	EMS-F-020
UC7 Create / Edit Event	Organizer	Create event (pending approval) and edit own events	EMS-F-007 to 009, 011
UC8 Cancel Event	Organizer, Email Service	Cancel event; notify attendees	EMS-F-012, 024
UC9 Check-in Attendees	Organizer	Mark attendance by registration ID / QR	EMS-F-026, 027

Use case	Actor(s)	Brief description	Related requirements
UC10 View Dashboard & Export CSV	Organizer	Statistics and CSV export for own events	EMS-F-028, 029
UC11 Approve / Reject Event	Admin	Review pending events	EMS-F-010
UC12 Manage Users & Categories	Admin	Activate, deactivate, change role; manage categories	EMS-F-031, 032
UC13 View Platform Summary	Admin	System-wide statistics	EMS-F-030
UC14 Send Notifications	Email Service	Deliver confirmations, reminders and notices	EMS-F-022 to 025

4.3 Data Model

The main entities and their relationships are shown below (field layouts in Appendix B).

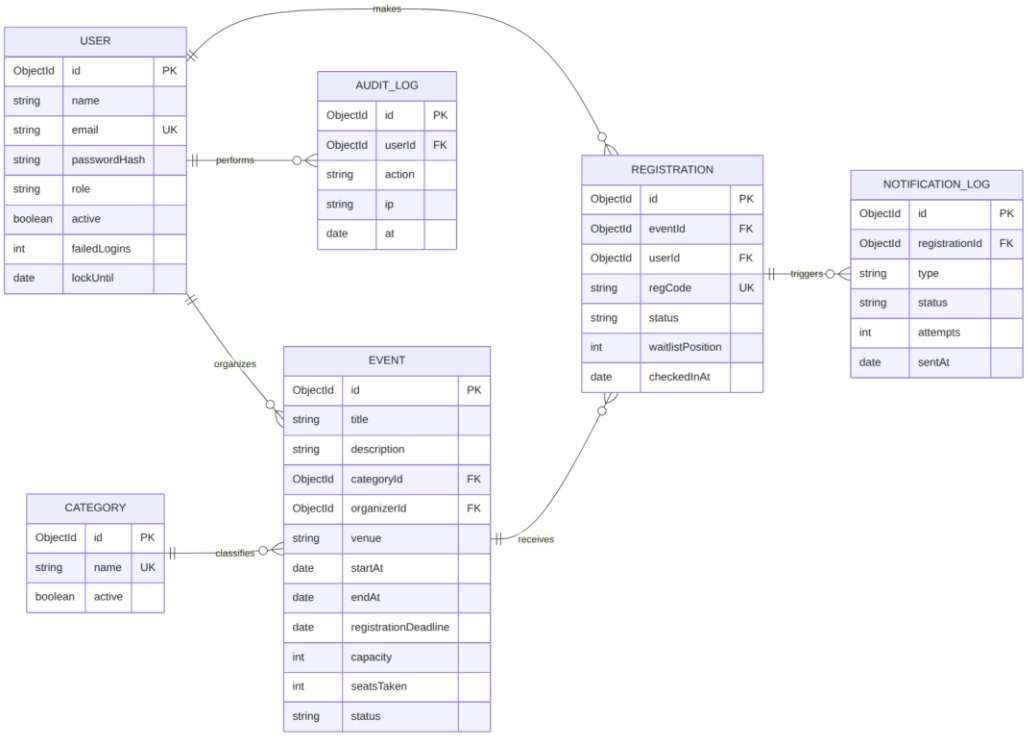


Figure 2: Entity-relationship model of EMS

5. System Features

5.1 F1: User Accounts & Authentication

5.1.1 Description and Priority

Lets visitors create accounts, log in securely, recover passwords, and enforces role-based access (Attendee, Organizer, Admin) across the whole system. **Priority: High.** Owner: Member 1.

5.1.2 Stimulus/Response Sequences

Stimulus (user action)	Response (system)
Visitor submits the registration form	System validates input, creates an Attendee account, and shows a success message
User submits valid credentials	System issues a session token and opens the role-specific home page
User submits wrong password 5 times in a row	System locks the account for 15 minutes and displays the lock message
User requests password reset	System emails a single-use reset link (valid 30 minutes)

5.1.3 Functional Requirements

ID	Requirement	Priority	Verification
EMS-F-001	The system shall allow a visitor to create an Attendee account by providing name, email and password (minimum 8 characters, with at least one letter and one digit); the email address shall be unique, and the account shall be created within 3 seconds of submission.	High	Test
EMS-F-002	The system shall authenticate a user by email and password and, on success, issue a session token that expires 60 minutes after issue.	High	Test
EMS-F-003	The system shall lock an account for 15 minutes after 5 consecutive failed login attempts and shall display the remaining lock time.	High	Test
EMS-F-004	The system shall support exactly three roles (Attendee, Organizer, Admin) and shall return HTTP 403 for any request to an endpoint that the requesting user's role is not permitted to use.	High	Test
EMS-F-005	The system shall email a single-use password reset link, valid for 30 minutes, to a registered email address on request, and shall invalidate the link after it is used once.	Medium	Test
EMS-F-006	The system shall allow a logged-in user to update their name and phone number, and shall reject phone numbers that are not 10 digits.	Low	Test

Implementation note for EMS-F-005 and EMS-F-006. Reset tokens are hashed and expire after 30 minutes. PUT /api/users/me supports profile updates. The client has no profile-edit screen, and invalid profile input returns a general validation message.

5.2 F2: Event Creation & Approval

5.2.1 Description and Priority

Organizers create and manage events; every new event waits for Admin approval before it becomes visible to attendees. **Priority: High.** Owner: Member 2.

5.2.2 Stimulus/Response Sequences

Stimulus (user action)	Response (system)
Organizer submits the create-event form	System validates fields and saves the event with status Pending Approval
Admin approves a pending event	System sets status to Published and notifies the organizer
Admin rejects a pending event with a reason	System sets status to Rejected and notifies the organizer with the reason
Organizer cancels own event	System sets status to Cancelled and triggers notices to all registered attendees

5.2.3 Functional Requirements

ID	Requirement	Priority	Verification
EMS-F-007	The system shall allow an Organizer to create an event by providing title (3-100 characters), description (up to 2000 characters), category, venue, start date-time, end date-time, registration deadline and capacity (integer from 1 to 10,000), and shall reject the submission if any mandatory field is missing.	High	Test
EMS-F-008	The system shall reject an event whose end date-time is not after its start date-time, or whose registration deadline is after its start date-time, and shall name the offending field in the error message.	High	Test
EMS-F-009	The system shall save every newly created event with status Pending Approval and shall not show it in public listings until an Admin approves it.	High	Test
EMS-F-010	The system shall allow an Admin to approve a pending event (status becomes Published) or reject it with a reason of at least 10 characters (status becomes Rejected), and shall notify the organizer of the decision.	High	Test
EMS-F-011	The system shall allow an Organizer to edit only the events they created, and shall reject a capacity value lower than the number of Confirmed registrations already held.	Medium	Test
EMS-F-012	The system shall allow an Organizer to cancel their own event before it starts, set its status to Cancelled, and queue a cancellation email to every Confirmed and Waitlisted attendee within 5 minutes.	High	Test

5.3 F3: Event Discovery

5.3.1 Description and Priority

Any visitor can browse, search, and filter published upcoming events and view event details including remaining seats. **Priority: High.** Owner: Member 3.

5.3.2 Stimulus/Response Sequences

Stimulus (user action)	Response (system)
Visitor opens the events page	System lists published upcoming events, 10 per page, sorted by start date
Visitor enters a keyword and/or filters	System returns only matching events
Visitor opens an event	System shows full details and remaining seats

5.3.3 Functional Requirements

ID	Requirement	Priority	Verification
EMS-F-013	The system shall display all Published events whose start date-time is in the future to any visitor, 10 events per page, sorted by earliest start date-time first.	High	Test
EMS-F-014	The system shall let a visitor search events by keyword (matched against title and description, case-insensitive) and filter by category, date range, and availability of seats, and shall return only events satisfying all selected criteria.	High	Test
EMS-F-015	The system shall display, for each Published event, its title, description, category, venue, start and end date-time, organizer name, and the number of remaining seats.	Medium	Test

5.4 F4: Event Registration & Waitlist

5.4.1 Description and Priority

Attendees register for events, join a waitlist when an event is full, cancel registrations, and track their status. Capacity is never exceeded. **Priority: High.** Owner: Member 3.

5.4.2 Stimulus/Response Sequences

Stimulus (user action)	Response (system)
Attendee clicks Register on an event with free seats	System creates a Confirmed registration with a unique registration ID and QR code
Attendee clicks Register on a full event	System offers and creates a Waitlisted registration
Attendee cancels a confirmed registration	System releases the seat and promotes the first waitlisted attendee
Attendee opens My Registrations	System lists all registrations with their status

5.4.3 Functional Requirements

ID	Requirement	Priority	Verification
EMS-F-016	The system shall allow a logged-in Attendee to register for a Published event before its registration deadline when seats remain, and shall generate a unique registration ID and a QR code containing that ID.	High	Test
EMS-F-017	The system shall reject a second active registration (Confirmed or Waitlisted) by the same Attendee for the same event with the message 'Already registered'.	High	Test
EMS-F-018	The system shall place an Attendee on a waitlist when the event is full and, when a Confirmed registration is cancelled, shall promote the earliest waitlisted Attendee to Confirmed (first-in-first-out) and notify them.	High	Test
EMS-F-019	The system shall allow an Attendee to cancel their own registration at any time before the event starts and shall release the seat immediately.	High	Test
EMS-F-020	The system shall show a logged-in Attendee all of their registrations with event name, date and status (Confirmed, Waitlisted, Cancelled, or Checked-in).	Medium	Test
EMS-F-021	The system shall ensure that the number of Confirmed registrations for an event never exceeds its capacity, even when 50 or more registration requests for the last seats arrive simultaneously.	High	Test

5.5 F5: Notifications

5.5.1 Description and Priority

Sends email notifications for confirmations, reminders, waitlist promotions, event changes and cancellations, and records delivery status. **Priority: Medium.** Owner: Member 4.

5.5.2 Stimulus/Response Sequences

Stimulus (user action)	Response (system)
Registration is confirmed	System emails the attendee a confirmation within 60 seconds
24 hours remain before an event starts	System emails a reminder to all confirmed attendees
Organizer changes date, time, or venue, or cancels	System emails all confirmed and waitlisted attendees
An email send fails	System retries up to 3 times and logs the final status

5.5.3 Functional Requirements

ID	Requirement	Priority	Verification
EMS-F-022	The system shall enqueue the registration confirmation email within 60 seconds of successful registration.	High	Test
EMS-F-023	The system shall send a reminder email to every Confirmed Attendee 24 hours (tolerance of plus or minus 15 minutes) before the event start time.	Medium	Test
EMS-F-024	The system shall email affected Attendees when an event's date, time or venue changes or the event is cancelled, and shall email the Organizer when an Admin approves or rejects their event.	Medium	Test

ID	Requirement	Priority	Verification
EMS-F-025	The system shall retry a failed email up to 3 times at 1-minute intervals and shall record the final delivery status (Sent or Failed) and number of attempts for every email.	Medium	Test

5.6 F6: Attendance Check-in

5.6.1 Description and Priority

Organizers record attendee presence at the event by registration ID or QR code. **Priority: Medium.** Owner: Member 4.

5.6.2 Stimulus/Response Sequences

Stimulus (user action)	Response (system)
Organizer enters or scans a registration ID during the event window	System marks the registration as Checked-in with a timestamp
Organizer scans a duplicate or non-confirmed registration	System rejects it with a clear message

5.6.3 Functional Requirements

ID	Requirement	Priority	Verification
EMS-F-026	The system shall allow an Organizer to mark an attendee as Checked-in by registration ID or QR code for their own event, only from 1 hour before the start time until the end time of the event, and shall record the check-in timestamp.	Medium	Test
EMS-F-027	The system shall reject a check-in for a registration that is not Confirmed or is already Checked-in, and shall display a message stating the reason.	Medium	Test

5.7 F7: Reports & Dashboards

5.7.1 Description and Priority

Organizers see registration and attendance statistics for their events and export attendee lists; Admins see platform-wide summaries. **Priority: Medium.** Owner: Member 4.

5.7.2 Stimulus/Response Sequences

Stimulus (user action)	Response (system)
Organizer opens an event dashboard	System shows registered, waitlisted, checked-in counts and attendance percentage
Organizer clicks Export CSV	System downloads the attendee list as a CSV file
Admin opens the platform summary	System shows event counts by status, users by role, and registrations over time

5.7.3 Functional Requirements

ID	Requirement	Priority	Verification
EMS-F-028	The system shall show an Organizer, for each of their events, the number of Confirmed, Waitlisted, Cancelled and Checked-in registrations and the attendance percentage (Checked-in divided by Confirmed).	Medium	Test
EMS-F-029	The system shall let an Organizer download the attendee list of their own event as a CSV file containing registration ID, name, email, status and check-in time.	Medium	Test
EMS-F-030	The system shall show an Admin the number of events by status, the number of users by role, and the number of registrations per day for the last 30 days.	Low	Test

5.8 F8: Administration

5.8.1 Description and Priority

Admins manage user accounts and roles and maintain the list of event categories. **Priority: Medium.** Owner: Member 1.

5.8.2 Stimulus/Response Sequences

Stimulus (user action)	Response (system)
Admin deactivates a user	System blocks that user's login and invalidates active sessions
Admin changes a user's role to Organizer	System updates the role; new permissions apply from the next request
Admin adds or disables a category	System updates the category list used by event forms and filters

5.8.3 Functional Requirements

ID	Requirement	Priority	Verification
EMS-F-031	The system shall allow an Admin to activate or deactivate any user account and to change a user's role; a deactivated user shall be unable to log in and existing sessions shall be rejected from the next request.	Medium	Test
EMS-F-032	The system shall allow an Admin to add, rename and disable event categories; a disabled category shall not be selectable for new events but shall remain visible on existing events.	Low	Test

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Requirement	Priority	Verification
EMS-NF-001	Event listing and search requests shall complete within 2 seconds for at least 95% of requests with 100 concurrent users and 5,000 events in the database.	High	Load test (JMeter)
EMS-NF-002	A registration request shall complete within 3 seconds for at least 95% of requests with 100 concurrent users.	High	Load test (JMeter)

6.2 Safety Requirements

ID	Requirement	Priority	Verification
EMS-SAF-001	The web client shall ask for explicit confirmation before cancelling an event, cancelling a registration, or deactivating a user, naming the item affected.	Medium	Test / demonstration
EMS-SAF-002	The system shall collect only the minimum personal data required for operation. Security metadata such as user ID and IP address may be retained for authentication and audit logging.	Medium	Inspection

6.3 Security Requirements

6.3.1 Security Objectives

ID	Objective	Description
SO-1	Confidentiality	Protect credentials, session tokens and personal data (name, email, phone) from disclosure to unauthorised parties.
SO-2	Integrity	Ensure events, registrations and attendance records can be created or changed only by authorised users through validated input.
SO-3	Availability	Keep the service usable under normal load and resist abuse such as brute-force login and request flooding.
SO-4	Accountability	Be able to determine who performed security-relevant actions and when.
SO-5	Least privilege	Give each role access only to the functions and data it needs.

6.3.2 Security Requirements

ID	Requirement	Objective	Verification
EMS-SEC-001	Passwords shall be stored only as bcrypt hashes with a cost factor of at least 10, and shall never appear in logs, API responses or error messages.	SO-1	Test / inspection
EMS-SEC-002	All communication between browser and server shall use HTTPS with TLS 1.2 or higher; plain HTTP requests shall be redirected to HTTPS.	SO-1	Test (TLS scan)
EMS-SEC-003	The server shall check the user's role and resource ownership on every protected request (for example, an Organizer may modify only own events); checks performed only in the web client shall not be relied upon.	SO-5, SO-2	Test

ID	Requirement	Objective	Verification
EMS-SEC-004	All request inputs shall be validated for type, length and format on the server, MongoDB operator characters in user input shall be rejected or stripped, and user-supplied text shall be output-encoded so that script content is not executed in the browser.	SO-2	Test (fuzz, injection)
EMS-SEC-005	The system shall limit each IP address to 100 API requests per minute and to 10 login attempts per minute, and shall respond with HTTP 429 when a limit is exceeded.	SO-3	Test
EMS-SEC-006	The system shall record user ID, action, IP address and timestamp for logins, failed logins, role changes, event approvals and rejections, event cancellations and check-ins, and shall retain these records for 90 days without storing passwords or tokens.	SO-4	Test / inspection
EMS-SEC-007	Session tokens shall be signed JSON Web Tokens with a 60-minute expiry, sent in an HttpOnly, Secure, SameSite=Strict cookie, and tokens that are expired, tampered with, or belong to a deactivated user shall be rejected with HTTP 401.	SO-1	Test
EMS-SEC-008	Secrets (database URI, token signing key, email credentials) shall be supplied through environment variables and shall not be committed to the repository, and the release build shall have no high or critical findings from 'npm audit'.	SO-1, SO-3	Inspection / scan

6.3.3 Identified threats (summary)

The threats considered are spoofing of identity (stolen or guessed credentials), tampering with events or registrations, disclosure of personal data, flooding or brute-force attacks, and privilege escalation by an Attendee. They are analysed in detail with STRIDE in SAD section 3.9 and validated in STP section 5.1.

6.4 Software Quality Attributes

Relative priority: security and correctness (no over-booking) are more important than visual polish; ease of use is more important than ease of learning for Organizers.

ID	Attribute	Requirement	Priority	Verification
EMS-NF-003	Availability	The deployed system shall be available at least 99% of the time during a 30-day measurement period, excluding announced maintenance windows.	Medium	Monitoring log
EMS-NF-004	Usability	At least 4 of 5 first-time test users shall be able to find an event and register for it within 3 minutes, using no more than 5 clicks from the events page, without instructions.	Medium	Usability test (5 users)
EMS-NF-005	Portability	The web client shall work without functional defects on the latest two versions of Chrome, Firefox and Edge, and shall render without horizontal scrolling at screen widths from 360 px to 1920 px.	Medium	Cross-browser test

ID	Attribute	Requirement	Priority	Verification
EMS-NF-006	Reliability	The database shall be backed up automatically at least once every 24 hours, and a restore from the most recent backup shall be demonstrated successfully at least once before final submission.	Medium	Inspection / restore drill
EMS-NF-007	Maintainability	Backend business-logic modules (services) shall have at least 70% statement coverage from automated unit tests, and the code base shall pass ESLint with zero errors.	Medium	Coverage report, lint
EMS-NF-008	Scalability	The system shall support 1,000 registered users and 200 events with no more than 10% degradation of the response times in EMS-NF-001 and EMS-NF-002, and the API tier shall keep no per-user state in memory.	Low	Load test, design review
EMS-NF-009	Accessibility	The home, event list, event detail and registration pages shall score at least 90 in the Lighthouse Accessibility audit.	Low	Lighthouse audit
EMS-NF-010	Data consistency	All date-times shall be stored in UTC and shown to users in Indian Standard Time (UTC+05:30).	Medium	Test

6.5 Business Rules and Domain Requirements

- **BR-01:** Only Published events are visible to Visitors and Attendees; Pending, Rejected and Cancelled events are visible only to their Organizer and Admins.
- **BR-02:** An Attendee may hold only one active (Confirmed or Waitlisted) registration per event.
- **BR-03:** Registration is open from the moment the event is Published until the registration deadline.
- **BR-04:** The waitlist is first-in-first-out; promotion happens automatically when a Confirmed registration is cancelled.
- **BR-05:** An Organizer may modify, cancel and view reports only for events they created; an Admin approves or rejects events but does not edit their content.
- **BR-06:** A cancelled event cannot be re-opened; the Organizer must create a new event.
- **BR-07:** Only Confirmed registrations can be checked in.
- **BR-08:** Every new account is an Attendee; only an Admin can grant the Organizer or Admin role.
- BR-09: Editing a published event returns it to Pending Approval for admin review.

- BR-10: Admin user changes must leave at least one active Admin. Checked-in bookings continue to occupy their seats.

- BR-11: Attendance is Checked-in divided by still-Confirmed plus Checked-in. The CSV retains the required attendee fields and adds event summary data.

7. Other Requirements

- **Database:** MongoDB with unique indexes on user email, registration ID, and (event, user) using a unique partial index that applies only to registrations with status Confirmed or Waitlisted; indexes on event start date, status and category to support EMS-NF-001.
- **Legal / privacy:** only the minimum personal data in EMS-SAF-002 is collected; users can request deletion of their account by contacting an Admin.
- **Internationalization:** English only in version 1.0; dates shown in IST.
- **Reuse:** open-source libraries only (React, Express, Mongoose, bcrypt, Nodemailer, Jest, Supertest, JMeter), checked for licences compatible with academic use.

Appendix A: Glossary

Term	Meaning
EMS	Event Management System
SRS / SAD / STP	Software Requirements Specification / Software Architecture and Design Specification / Software Test Plan
Attendee	Registered user who browses and registers for events
Organizer	User who creates and manages events
Admin	Administrator who approves events and manages users and categories
Visitor	A person who is not logged in
Registration ID	Unique code (for example REG-7F3K9Q) that identifies a registration and is encoded in the QR code
Waitlist	Ordered queue of attendees waiting for a seat in a full event
RBAC	Role-Based Access Control
JWT	JSON Web Token, used to keep a user logged in
REST	Style of web API used between client and server
TLS / HTTPS	Transport Layer Security / HTTP over TLS
SMTP	Protocol used to send email
IST / UTC	Indian Standard Time (UTC+05:30) / Coordinated Universal Time
Pending Approval / Published / Rejected / Cancelled	Event statuses
Confirmed / Waitlisted / Cancelled / Checked-in	Registration statuses

Term	Meaning
Capacity	Maximum number of Confirmed registrations for an event

Appendix B: Field Layouts

B.1 User

Field	Length	Data type	Description	Mandatory
name	3-60	String	Full name	Y
email	up to 100	String (email)	Unique login id	Y
password	8-64	String	Entered by user; stored only as bcrypt hash	Y
phone	10	Numeric	Mobile number	N
role	-	Enum	Attendee / Organizer / Admin	Y
active	-	Boolean	False when deactivated	Y

B.2 Event

Field	Length	Data type	Description	Mandatory
title	3-100	String	Event title	Y
description	up to 2000	String	Event details	Y
category	-	Reference	One of the active categories	Y
venue	up to 100	String	Location	Y
startAt / endAt	-	Date-time (UTC)	Start and end; end must be after start	Y
registrationDeadline	-	Date-time (UTC)	Must not be after startAt	Y
capacity	1-10000	Integer	Maximum Confirmed registrations	Y
status	-	Enum	Pending Approval / Published / Rejected / Cancelled	Y
rejectionReason	10-300	String	Required when status is Rejected	N

B.3 Registration

Field	Length	Data type	Description	Mandatory
regCode	10	Alphanumeric	Unique registration ID, also encoded in QR code	Y
event / user	-	Reference	Event and Attendee	Y
status	-	Enum	Confirmed / Waitlisted / Cancelled / Checked-in	Y
waitlistPosition	-	Integer	Position for Waitlisted registrations	N

Field	Length	Data type	Description	Mandatory
checkedInAt	-	Date-time (UTC)	Time of check-in	N

B.4 Report requirements

Attendee list (CSV)	Organizer dashboard	Admin summary
Registration ID	Confirmed count	Events by status
Name	Waitlisted count	Users by role
Email	Cancelled count	Registrations per day (30 days)
Status	Checked-in count	
Check-in time	Attendance %	

Appendix C: Requirement Traceability Matrix

Architecture references use component IDs from the SAD (section 3.4); design references point to SAD sections; test case IDs are defined in the STP (section 13).

#	Req. ID	Brief description	Arch.	Design ref.	Code file (planned)	Test case ID	System test ID
1	EMS-F-001	Attendee registration	C3	SAD 4.3.1 POST /auth/register	backend/src/modules/auth/auth.controller.js	TC-AUTH-01, TC-AUTH-02	-
2	EMS-F-002	Login and session token	C3	SAD 4.3.1 POST /auth/login	backend/src/modules/auth/auth.service.js	TC-AUTH-03	TC-AUTH-03
3	EMS-F-003	Account lockout	C3	SAD 4.3.1 POST /auth/login	backend/src/modules/auth/lockout.js	TC-AUTH-04	-
4	EMS-F-004	Role-based access control	C2, C3	SAD 3.9, 4.3 (all endpoints)	backend/src/middleware/rbac.js	TC-AUTH-05	-
5	EMS-F-005	Password reset	C3, C6	SAD 4.3.1 POST /auth/forgot-password, /auth/reset-password	backend/src/modules/auth/reset.service.js	TC-AUTH-06	-
6	EMS-F-006	Profile update	C3	SAD 4.3.1 PUT /users/me	backend/src/modules/auth/profile.controller.js	TC-AUTH-07	-
7	EMS-F-007	Create event	C4	SAD 4.3.2 POST /events; SD-2	backend/src/modules/events/event.controller.js	TC-EVT-01, TC-EVT-02	TC-EVT-01
8	EMS-F-008	Event date validation	C4	SAD 4.3.2 POST /events; SD-2	backend/src/modules/events/event.validator.js	TC-EVT-02	-
9	EMS-F-009	Pending approval status	C4	SAD 4.3.2; SD-2	backend/src/modules/events/event.service.js	TC-EVT-01	TC-EVT-01
10	EMS-F-010	Admin approval / rejection	C4, C6	SAD 4.3.2 PATCH /events/{id}/approval; SD-2	backend/src/modules/events/approval.controller.js	TC-EVT-03	TC-EVT-03
11	EMS-F-011	Edit own event	C4	SAD 4.3.2 PATCH /events/{id}	backend/src/modules/events/event.service.js	TC-EVT-04	-
12	EMS-F-012	Cancel event	C4, C6	SAD 4.3.2 POST /events/{id}/cancel	backend/src/modules/events/cancel.service.js	TC-EVT-05	TC-EVT-05

#	Req. ID	Brief description	Arch.	Design ref.	Code file (planned)	Test case ID	System test ID
13	EMS-F-013	Browse published events	C4	SAD 4.3.2 GET /events	backend/src/modules/events/event.query.js	TC-SRC-01	TC-SRC-01
14	EMS-F-014	Search and filter	C4	SAD 4.3.2 GET /events?q=&category=&from=&to=&available=	backend/src/modules/events/event.query.js	TC-SRC-01	TC-SRC-01
15	EMS-F-015	Event details	C4	SAD 4.3.2 GET /events/{id}	backend/src/modules/events/event.controller.js	TC-SRC-02	-
16	EMS-F-016	Register for event	C5	SAD 4.3.3 POST /events/{id}/registrations; SD-1	backend/src/modules/registrations/registration.service.js	TC-REG-01, TC-REG-02	TC-REG-01
17	EMS-F-017	No duplicate registration	C5	SAD 4.3.3; SD-1	backend/src/modules/registrations/registration.service.js	TC-REG-02	-
18	EMS-F-018	Waitlist with FIFO promotion	C5, C6	SAD 4.3.3; SD-1, SD-3	backend/src/modules/registrations/waitlist.service.js	TC-REG-03, TC-REG-04	TC-REG-03, TC-REG-04
19	EMS-F-019	Cancel registration	C5	SAD 4.3.3 DELETE /registrations/{id}; SD-3	backend/src/modules/registrations/registration.service.js	TC-REG-04	TC-REG-04
20	EMS-F-020	My registrations	C5	SAD 4.3.3 GET /registrations/me	backend/src/modules/registrations/registration.controller.js	TC-REG-05	-
21	EMS-F-021	Capacity never exceeded	C5, C8	SAD 3.5, SD-1 (atomic seat update)	backend/src/modules/registrations/seat.repository.js	TC-REG-06	-
22	EMS-F-022	Registration confirmation email	C6	SAD 4.4; SD-1	backend/src/modules/notifications/mailer.service.js	TC-NTF-01	TC-NTF-01
23	EMS-F-023	24-hour reminder	C6	SAD 4.4 (scheduler)	backend/src/modules/notifications/reminder.job.js	TC-NTF-02	-
24	EMS-F-024	Change / cancellation / decision notices	C6	SAD 4.3.2; SD-2	backend/src/modules/notifications/templates/	TC-EVT-05	TC-EVT-05
25	EMS-F-025	Retry and delivery log	C6, C8	SAD 4.4	backend/src/modules/notifications/retry.js	TC-NTF-03	-
26	EMS-F-026	Check-in attendee	C7	SAD 4.3.4 POST /events/{id}/checkin	backend/src/modules/attendance/checkin.service.js	TC-CHK-01	TC-CHK-01
27	EMS-F-027	Reject invalid check-in	C7	SAD 4.3.4	backend/src/modules/attendance/checkin.service.js	TC-CHK-02	-
28	EMS-F-028	Organizer dashboard	C7	SAD 4.3.4 GET /events/{id}/report	backend/src/modules/reports/dashboard.service.js	TC-RPT-01	TC-RPT-01
29	EMS-F-029	Export attendee CSV	C7	SAD 4.3.4 GET /events/{id}/attendees.csv	backend/src/modules/reports/export.service.js	TC-RPT-01	TC-RPT-01
30	EMS-F-030	Admin platform summary	C7	SAD 4.3.4 GET /admin/summary	backend/src/modules/reports/summary.service.js	TC-RPT-02	-
31	EMS-F-031	Manage users and roles	C3	SAD 4.3.1 PATCH /admin/users/{id}	backend/src/modules/admin/user.admin.controller.js	TC-ADM-01	-
32	EMS-F-032	Manage categories	C4	SAD 4.3.2 /admin/categories	backend/src/modules/admin/category.controller.js	TC-ADM-02	-

#	Req. ID	Brief description	Arch.	Design ref.	Code file (planned)	Test case ID	System test ID
33	EMS-NF-001	Search response time	C2, C4, C8	SAD 3.5, 3.6 (indexes)	backend/src/models/*.js (indexes)	TC-PERF-01	TC-PERF-01
34	EMS-NF-002	Registration response time	C5, C8	SAD SD-1	backend/src/modules/registrations/	TC-PERF-02	TC-PERF-02
35	EMS-NF-003	Availability	All	SAD 4.4 (health check)	backend/src/app.js (/health)	TC-AVL-01	TC-AVL-01
36	EMS-NF-004	Ease of registration	C1	SAD 4.5	frontend/src/pages/	TC-USB-01	TC-USB-01
37	EMS-NF-005	Browser and device support	C1	SAD 4.5	frontend/src/	TC-CMP-01	TC-CMP-01
38	EMS-NF-006	Backup and recovery	C9	SAD 3.7	ops/backup.sh	TC-AVL-01	TC-AVL-01
39	EMS-NF-007	Test coverage and code quality	C2-C8	SAD 3.5	backend/tests/, .eslintrc	TC-MNT-01	-
40	EMS-NF-008	Capacity of the system	C2, C9	SAD 3.5 (no per-user state in API tier)	backend/src/app.js	TC-PERF-01	TC-PERF-01
41	EMS-NF-009	Accessibility	C1	SAD 4.5	frontend/src/	TC-CMP-01	TC-CMP-01
42	EMS-NF-010	Time zone handling	C4, C8	SAD 3.6	backend/src/utils/time.js	TC-TZ-01	-
43	EMS-SAF-001	Confirm destructive actions	C1	SAD 4.5	frontend/src/components/ConfirmDialog.jsx	TC-SAF-01	TC-SAF-01
44	EMS-SAF-002	Minimal personal data	C8, C9	SAD 3.9	backend/src/models/user.model.js	TC-SAF-01	TC-SAF-01
45	EMS-SEC-001	Password hashing	C3	SAD 3.9	backend/src/modules/auth/password.js	TC-SEC-01	-
46	EMS-SEC-002	Encrypted transport	C2	SAD 3.9	deployment config / reverse proxy	TC-SEC-02	TC-SEC-02
47	EMS-SEC-003	Server-side authorization	C2	SAD 3.9, 4.3	backend/src/middleware/rbac.js, ownership.js	TC-AUTH-05, TC-EVT-04	-
48	EMS-SEC-004	Input validation and output encoding	C2	SAD 3.9, 4.4	backend/src/middleware/validate.js	TC-SEC-03	-
49	EMS-SEC-005	Rate limiting	C2	SAD 3.9	backend/src/middleware/rateLimit.js	TC-AUTH-04, TC-SEC-04	-
50	EMS-SEC-006	Audit logging	C10	SAD 3.9, 4.4	backend/src/modules/audit/audit.service.js	TC-SEC-05	-
51	EMS-SEC-007	Secure session tokens	C2, C3	SAD 3.9	backend/src/middleware/auth.js	TC-SEC-06	-
52	EMS-SEC-008	Secrets and dependencies	All	SAD 3.9	.gitignore, .github/workflows/ci.yml	TC-SEC-07	TC-SEC-07